

Open the document **evidence.doc**

Make sure that your name, centre number and candidate number will appear on every page of this document. This document must contain your answers to each question.

Save this evidence document in your work area as:

evidence_ followed by your centre number_candidate number, for example: evidence_zz999_9999

A class declaration can be used to declare a record.

If the programming language used does not support arrays, a list can be used instead.

Candidate source files are used to answer Question 3. The files are: DataToAdd.txt and SecondData.txt.

1 A 1D array, *TheData*, stores the following data:

20	3	4	8	12	99	4	26	4
----	---	---	---	----	----	---	----	---

- (a) Write a program to declare the array as a local variable, and initialise the values with the data given. Give the array the identifier *TheData*.

Save your program as **question1**.

Copy and paste the program code into **part 1(a)** in the evidence document.

[2]

- (b) Pseudocode for an insertion sort is shown below with **three** pieces of code missing.

```

PROCEDURE InsertionSort(TheData[]:INTEGER)
  FOR Count ← FirstElement TO .....
    DataToInsert ← TheData[i]
    Inserted ← 0
    NextValue ← Count - 1
    WHILE (NextValue >= 0 AND ..... <> 1)
      IF (DataToInsert < TheData[NextValue]) THEN
        TheData[NextValue + 1] ← TheData[NextValue]
        NextValue ← NextValue - 1
        TheData[NextValue + 1] ← .....
      ELSE
        Inserted ← 1
      ENDIF
    ENDWHILE
  NEXT
ENDPROCEDURE

```

In the program you wrote for **part 1(a)**, write the procedure *InsertionSort* to perform an insertion sort on the data in *TheData*. Pass the array you declared in **part 1(a)** by reference. Follow the pseudocode given and insert the missing pieces of code in your program.

Save your program.

Copy and paste the program code into **part 1(b)** in the evidence document.

[7]

- (c) Write a procedure to output all the contents of the array `TheData` to the screen. The procedure should use iteration. Pass the array into the procedure as a parameter.

Save your program.

Copy and paste the program code into **part 1(c)** in the evidence document.

[3]

- (d) (i) The main program needs to output the data before the data has been sorted, and after the data has been sorted.

Use the subroutines you declared in **part (b)** and **part (c)**.

Edit the main program so it:

- outputs all the data in `TheData` before sorting
- sorts the data in `TheData`
- outputs the data after sorting
- outputs appropriate headings to identify the outputs before and after sorting.

Save your program.

Copy and paste the program code into **part 1(d)(i)** in the evidence document.

[3]

- (ii) Test your program and take a screenshot to show the result.

Save your program.

Copy and paste the screenshot into **part 1(d)(ii)** in the evidence document.

[2]

- (e) (i) Write a function that:

- takes a whole number as input from the user
- if the number is in `TheData` outputs 'found' and returns true
- if the number is not in `TheData` outputs 'not found' and returns false.

Save your program.

Copy and paste the program code into **part 1(e)(i)** in the evidence document.

[6]

- (ii) Test the function using the number 8 as input, then the number 9 as input.

Take a screenshot of the results of each test, making sure the input is visible in the screenshot.

Save your program.

Copy and paste the screenshots into **part 1(e)(ii)** in the evidence document.

[2]